UNICESUMAR
CURSO DE ENGENHARIA DE SOFTWARE

JUAN GUIMARÃES MENDONÇA
MURILO CONSTANCIO AMADEU
VITOR GARRIDO DA SILVA

SISTEMA DE PLANEJAMENTO E ACOMPANHAMENTO DE AÇÕES DO PROGRAMA SAÚDE NA ESCOLA

MARINGÁ
2026

1. Implementação completa em linguagem C

O código abaixo contempla:

- Cadastro de ações.
- Listagem de ações.
- Consulta pelo código.
- Atualização da situação.
- Registro de participação.
- Geração de resumo.
- Validação de códigos duplicados.
- Limite de 100 ações.
- Validação de quantidades negativas.
- Menu com repetição até o usuário escolher sair.

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>
#include <errno.h>

#define MAX_ACOES          100
#define TAM_TEXTO          100
#define TAM_DATA           11      /* dd/mm/aaaa + '\0' */
#define MAX_CODIGO         999999
#define MAX_PARTICIPANTES  1000000
#define ARQUIVO_DADOS      "acoes.dat"

typedef enum {
    PLANEJADA = 1,
    REALIZADA = 2,
    CANCELADA = 3
} Situacao;

typedef struct {
    int      codigo;
    char     nome[TAM_TEXTO];
    char     escola[TAM_TEXTO];
    char     tema[TAM_TEXTO];
    char     data[TAM_DATA];
    char     publico[TAM_TEXTO];
    char     responsavel[TAM_TEXTO];
    int      participantesPrevistos;
    int      participantesEfetivos;
    Situacao situacao;
```

```
} Acao;

static Acao acoes[MAX_ACOES];
static int  totalAcoes = 0;

static const char *NOMES_SITUACAO[] = { "", "Planejada", "Realizada", "Cancelada" };

/* ---------------------------------------------------------------- */
/*  Entrada de dados                                                */
/* ---------------------------------------------------------------- */

/* Le uma linha. Retorna 1 se a linha era maior que o buffer (foi truncada
   e o restante descartado), 0 caso contrario. Encerra o programa em EOF
   (evita loop infinito quando a entrada acaba, ex.: Ctrl+D / Ctrl+Z). */
static int lerLinha(const char *mensagem, char *destino, size_t tamanho) {
    printf("%s", mensagem);

    if (fgets(destino, (int) tamanho, stdin) == NULL) {
        printf("\nEntrada encerrada. Saindo...\n");
        exit(0);
    }

    size_t len = strlen(destino);

    if (len > 0 && destino[len - 1] == '\n') {
        destino[len - 1] = '\0';
        return 0;
    }

    /* Sem '\n': a linha nao coube no buffer. Descarta o resto. */
    int c;
    int truncou = 0;
    while ((c = getchar()) != '\n' && c != EOF) {
        truncou = 1;
    }
    return truncou;
}

/* Limpa o terminal (Windows: cls | Linux/macOS: clear). */
static void limparTela(void) {
#ifdef _WIN32
    system("cls");
#else
    system("clear");
#endif
}

/* Espera o usuario apertar Enter, para dar tempo de ler o resultado
   antes de a tela ser limpa. */
static void pausar(void) {
```

```
    char lixo[8];
    lerLinha("\nPressione Enter para continuar...", lixo, sizeof(lixo));
}

static void aparar(char *s) {
    size_t ini = 0;
    size_t fim = strlen(s);

    while (s[ini] && isspace((unsigned char) s[ini])) ini++;
    while (fim > ini && isspace((unsigned char) s[fim - 1])) fim--;

    memmove(s, s + ini, fim - ini);
    s[fim - ini] = '\0';
}

static void lerTexto(const char *mensagem, char *destino, size_t tamanho) {
    while (1) {
        if (lerLinha(mensagem, destino, tamanho)) {
            printf("Texto muito longo (maximo %zu caracteres).\n", tamanho - 1);
            continue;
        }

        aparar(destino);

        if (destino[0] == '\0') {
            printf("Campo obrigatorio. Tente novamente.\n");
            continue;
        }
        return;
    }
}

static int lerInteiro(const char *mensagem, int minimo, int maximo) {
    char entrada[32];
    char *fim;
    long valor;

    while (1) {
        if (lerLinha(mensagem, entrada, sizeof(entrada))) {
            printf("Entrada muito longa.\n");
            continue;
        }

        errno = 0;
        valor = strtol(entrada, &fim, 10);

        if (fim == entrada || errno == ERANGE) {
            printf("Digite um numero inteiro valido.\n");
            continue;
        }
```

```
        while (isspace((unsigned char) *fim)) fim++;

        if (*fim != '\0') {
            printf("Digite um numero inteiro valido.\n");
            continue;
        }

        if (valor < minimo || valor > maximo) {
            printf("Valor fora do intervalo permitido (%d a %d).\n", minimo, maximo);
            continue;
        }

        return (int) valor;
    }
}

static int lerConfirmacao(const char *mensagem) {
    char entrada[8];

    while (1) {
        lerLinha(mensagem, entrada, sizeof(entrada));
        aparar(entrada);

        if (entrada[0] == 's' || entrada[0] == 'S') return 1;
        if (entrada[0] == 'n' || entrada[0] == 'N') return 0;

        printf("Responda com 's' ou 'n'.\n");
    }
}

/* ---------------------------------------------------------------- */
/*  Validacao de data (dd/mm/aaaa)                                  */
/* ---------------------------------------------------------------- */

static int dataValida(int d, int m, int a) {
    int dias[] = { 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31 };

    if (a < 2000 || a > 2100 || m < 1 || m > 12 || d < 1) return 0;

    if (m == 2 && ((a % 4 == 0 && a % 100 != 0) || a % 400 == 0)) {
        dias[1] = 29;
    }

    return d <= dias[m - 1];
}

static int formatoDataOk(const char *s) {
    if (strlen(s) != 10) return 0;
```

```
    for (int i = 0; i < 10; i++) {
        if (i == 2 || i == 5) {
            if (s[i] != '/') return 0;
        } else if (!isdigit((unsigned char) s[i])) {
            return 0;
        }
    }
    return 1;
}

static void lerData(const char *mensagem, char *destino) {
    while (1) {
        lerTexto(mensagem, destino, TAM_DATA);

        if (formatoDataOk(destino) &&
            dataValida(atoi(destino), atoi(destino + 3), atoi(destino + 6))) {
            return;
        }
        printf("Data invalida. Use o formato dd/mm/aaaa (ex.: 25/11/2026).\n");
    }
}

/* ---------------------------------------------------------------- */
/*  Persistencia                                                    */
/* ---------------------------------------------------------------- */

static int salvarDados(void) {
    FILE *f = fopen(ARQUIVO_DADOS, "wb");

    if (f == NULL) {
        printf("Aviso: nao foi possivel salvar em '%s'.\n", ARQUIVO_DADOS);
        return 0;
    }

    int ok = fwrite(&totalAcoes, sizeof(int), 1, f) == 1 &&
             fwrite(acoes, sizeof(Acao), (size_t) totalAcoes, f) == (size_t) totalAcoes;

    fclose(f);

    if (!ok) printf("Aviso: erro ao gravar os dados.\n");
    return ok;
}

static void carregarDados(void) {
    FILE *f = fopen(ARQUIVO_DADOS, "rb");
    int total;

    if (f == NULL) return; /* primeira execucao: ainda nao existe arquivo */

    if (fread(&total, sizeof(int), 1, f) == 1 && total >= 0 && total <= MAX_ACOES &&
```

```
        fread(acoes, sizeof(Acao), (size_t) total, f) == (size_t) total) {
        totalAcoes = total;
        printf("%d acao(oes) carregada(s) de '%s'.\n", totalAcoes, ARQUIVO_DADOS);
    } else {
        printf("Aviso: arquivo '%s' invalido. Iniciando sem dados.\n", ARQUIVO_DADOS);
    }

    fclose(f);
}

/* ------------------------------------------------------------- */
/*  Utilitarios                                                  */
/* ------------------------------------------------------------- */

static int buscarAcao(int codigo) {
    for (int i = 0; i < totalAcoes; i++) {
        if (acoes[i].codigo == codigo) return i;
    }
    return -1;
}

/* Pede um codigo e devolve o indice da acao, ou -1 se nao existir. */
static int pedirAcaoExistente(void) {
    int codigo = lerInteiro("Codigo da acao: ", 1, MAX_CODIGO);
    int indice = buscarAcao(codigo);

    if (indice == -1) printf("Acao nao encontrada.\n");
    return indice;
}

static void imprimirAcao(const Acao *a) {
    printf("\nCodigo: %d\n", a->codigo);
    printf("Nome: %s\n", a->nome);
    printf("Escola: %s\n", a->escola);
    printf("Tema: %s\n", a->tema);
    printf("Data: %s\n", a->data);
    printf("Publico-alvo: %s\n", a->publico);
    printf("Responsavel: %s\n", a->responsavel);
    printf("Participantes previstos: %d\n", a->participantesPrevistos);
    printf("Participantes efetivos: %d\n", a->participantesEfetivos);
    printf("Situacao: %s\n", NOMES_SITUACAO[a->situacao]);

    if (a->situacao == REALIZADA && a->participantesPrevistos > 0) {
        printf("Adesao: %.1f%%\n",
               100.0 * a->participantesEfetivos / a->participantesPrevistos);
    }
}

/* ------------------------------------------------------------- */
/*  Funcionalidades                                              */
```

```
/* ---------------------------------------------------------------- */

static void exibirMenu(void) {
    printf("\n========================================\n");
    printf("       SISTEMA PSE - SAUDE NA ESCOLA\n");
    printf("========================================\n");
    printf("1 - Cadastrar acao\n");
    printf("2 - Listar todas as acoes\n");
    printf("3 - Listar acoes por situacao\n");
    printf("4 - Consultar acao\n");
    printf("5 - Atualizar situacao\n");
    printf("6 - Registrar participacao\n");
    printf("7 - Remover acao\n");
    printf("8 - Gerar resumo\n");
    printf("0 - Sair\n");
    printf("========================================\n");
}

static void cadastrarAcao(void) {
    Acao nova;

    if (totalAcoes >= MAX_ACOES) {
        printf("\nLimite de %d acoes atingido.\n", MAX_ACOES);
        return;
    }

    printf("\n--- CADASTRO DE ACAO ---\n");

    nova.codigo = lerInteiro("Codigo da acao: ", 1, MAX_CODIGO);

    if (buscarAcao(nova.codigo) != -1) {
        printf("Erro: codigo ja cadastrado.\n");
        return;
    }

    lerTexto("Nome da acao: ", nova.nome, sizeof(nova.nome));
    lerTexto("Escola: ", nova.escola, sizeof(nova.escola));
    lerTexto("Tema: ", nova.tema, sizeof(nova.tema));
    lerData("Data prevista (dd/mm/aaaa): ", nova.data);
    lerTexto("Publico-alvo: ", nova.publico, sizeof(nova.publico));
    lerTexto("Responsavel: ", nova.responsavel, sizeof(nova.responsavel));

    nova.participantesPrevistos =
        lerInteiro("Quantidade prevista de participantes: ", 0, MAX_PARTICIPANTES);
    nova.participantesEfetivos = 0;
    nova.situacao = PLANEJADA;

    acoes[totalAcoes++] = nova;
    salvarDados();
```

```
    printf("\nAcao cadastrada com sucesso! Situacao inicial: Planejada\n");
}

static void listarAcoes(void) {
    printf("\n--- LISTA DE ACOES ---\n");

    if (totalAcoes == 0) {
        printf("Nenhuma acao cadastrada.\n");
        return;
    }

    for (int i = 0; i < totalAcoes; i++) {
        imprimirAcao(&acoes[i]);
    }
}

static void listarPorSituacao(void) {
    int encontradas = 0;

    printf("\n--- LISTAR POR SITUACAO ---\n");
    printf("1 - Planejada\n2 - Realizada\n3 - Cancelada\n");

    int situacao = lerInteiro("Situacao: ", 1, 3);

    for (int i = 0; i < totalAcoes; i++) {
        if (acoes[i].situacao == (Situacao) situacao) {
            imprimirAcao(&acoes[i]);
            encontradas++;
        }
    }

    if (encontradas == 0) {
        printf("\nNenhuma acao %s.\n", NOMES_SITUACAO[situacao]);
    }
}

static void consultarAcao(void) {
    printf("\n--- CONSULTAR ACAO ---\n");

    int indice = pedirAcaoExistente();
    if (indice == -1) return;

    printf("\nAcao encontrada!");
    imprimirAcao(&acoes[indice]);
}

static void atualizarSituacao(void) {
    printf("\n--- ATUALIZAR SITUACAO ---\n");

    int indice = pedirAcaoExistente();
```

```
    if (indice == -1) return;

    Acao *a = &acoes[indice];

    printf("\nSituacao atual: %s\n", NOMES_SITUACAO[a->situacao]);
    printf("1 - Planejada\n2 - Realizada\n3 - Cancelada\n");

    Situacao nova = (Situacao) lerInteiro("Nova situacao: ", 1, 3);

    if (nova == a->situacao) {
        printf("A acao ja esta com essa situacao.\n");
        return;
    }

    if (a->situacao == REALIZADA && a->participantesEfetivos > 0 &&
        !lerConfirmacao("Isso apagara a participacao registrada. Continuar? (s/n): ")) {
        printf("Operacao cancelada.\n");
        return;
    }

    a->situacao = nova;
    a->participantesEfetivos = 0;

    if (nova == REALIZADA) {
        a->participantesEfetivos =
            lerInteiro("Quantidade efetiva de participantes: ", 0, MAX_PARTICIPANTES);
    }

    salvarDados();
    printf("Situacao atualizada com sucesso!\n");
}

static void registrarParticipacao(void) {
    printf("\n--- REGISTRAR PARTICIPACAO ---\n");

    int indice = pedirAcaoExistente();
    if (indice == -1) return;

    if (acoes[indice].situacao != REALIZADA) {
        printf("A participacao so pode ser registrada em acoes realizadas.\n");
        return;
    }

    acoes[indice].participantesEfetivos =
        lerInteiro("Quantidade efetiva de participantes: ", 0, MAX_PARTICIPANTES);

    salvarDados();
    printf("Participacao registrada com sucesso!\n");
}
```

```
static void removerAcao(void) {
    printf("\n--- REMOVER ACAO ---\n");

    int indice = pedirAcaoExistente();
    if (indice == -1) return;

    printf("Acao: %s (%s)\n", acoes[indice].nome, acoes[indice].escola);

    if (!lerConfirmacao("Confirma a remocao? (s/n): ")) {
        printf("Operacao cancelada.\n");
        return;
    }

    memmove(&acoes[indice], &acoes[indice + 1],
            (size_t) (totalAcoes - indice - 1) * sizeof(Acao));
    totalAcoes--;

    salvarDados();
    printf("Acao removida com sucesso!\n");
}

static void gerarResumo(void) {
    int contagem[4] = { 0 };
    long previstos = 0;
    long efetivos = 0;
    double percentual = 0.0;

    for (int i = 0; i < totalAcoes; i++) {
        contagem[acoes[i].situacao]++;

        if (acoes[i].situacao == REALIZADA) {
            previstos += acoes[i].participantesPrevistos;
            efetivos  += acoes[i].participantesEfetivos;
        }
    }

    if (previstos > 0) {
        percentual = 100.0 * efetivos / previstos;
    }

    printf("\n--- RESUMO GERAL ---\n");
    printf("Total de acoes cadastradas: %d\n", totalAcoes);
    printf("Acoes planejadas: %d\n", contagem[PLANEJADA]);
    printf("Acoes realizadas: %d\n", contagem[REALIZADA]);
    printf("Acoes canceladas: %d\n", contagem[CANCELADA]);
    printf("Participantes previstos nas acoes realizadas: %ld\n", previstos);
    printf("Participantes efetivos: %ld\n", efetivos);
    printf("Percentual de participacao: %.2f%%\n", percentual);
}
```

```
/* ------------------------------------------------------------------ */

int main(void) {
    int opcao;

    carregarDados();
    pausar(); /* permite ler a mensagem de carregamento */

    do {
        limparTela();
        exibirMenu();
        opcao = lerInteiro("Digite uma opcao: ", 0, 8);

        limparTela();

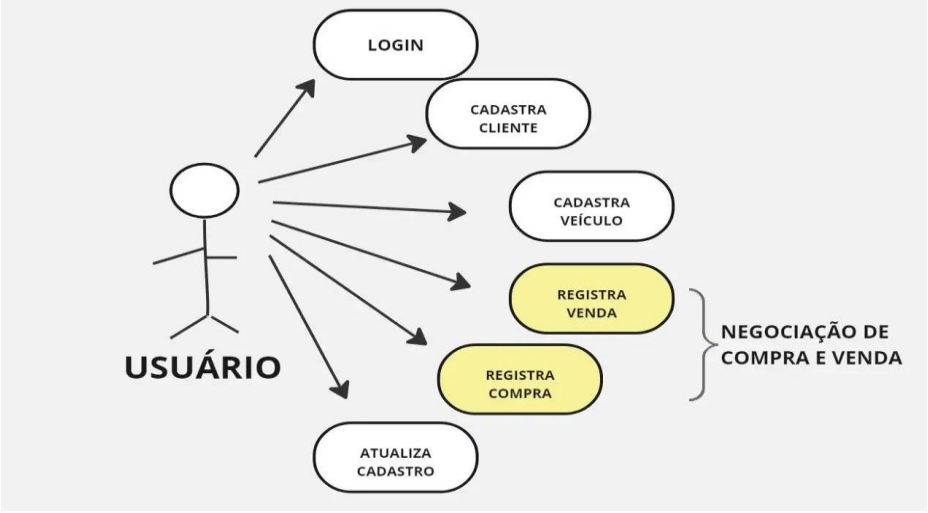
        switch (opcao) {
            case 1: cadastrarAcao();        break;
            case 2: listarAcoes();          break;
            case 3: listarPorSituacao();    break;
            case 4: consultarAcao();        break;
            case 5: atualizarSituacao();    break;
            case 6: registrarParticipacao();break;
            case 7: removerAcao();          break;
            case 8: gerarResumo();          break;
            case 0: printf("\nPrograma encerrado. Obrigado!\n"); break;
        }

        if (opcao != 0) pausar();
    } while (opcao != 0);

    return 0;
}
```

2. Diagrama de casos de uso

O ator principal será o usuário do sistema. Ele poderá executar as seis funcionalidades.




```
                SISTEMA PSE
                  +---------------------------------+
                  |                                 |
                  |   (Cadastrar ação)              |
                  |                                 |
                  |   (Listar ações)                |
                  |                                 |
USUÁRIO ---------- (Consultar ação)                 |
                  |                                 |
                  |   (Atualizar situação)          |
                  |                                 |
                  |   (Registrar participação)      |
                  |                                 |
                  |   (Gerar resumo)                |
                  |                                 |
                  +---------------------------------+
```

O ator deve ficar fora do retângulo, ligado por linhas aos seis casos de uso.

3. Descrição dos casos de uso

Esta parte é importante porque vale um dos critérios de Engenharia de Software.

UC01 — Cadastrar ação

- **Objetivo:** cadastrar uma nova ação do Programa Saúde na Escola.
- **Ator:** usuário.
- **Pré-condições:** o sistema deve estar em execução e possuir espaço disponível no vetor.
- **Fluxo principal:**
    1. O usuário seleciona a opção 1.
    2. O sistema solicita o código da ação.
    3. O sistema verifica se o código já existe.
    4. O usuário informa os dados da ação.
    5. O sistema valida as informações.
    6. O sistema define a situação inicial como Planejada.
    7. O sistema armazena a ação.

8. O sistema confirma o cadastro.

- **Fluxos alternativos:** código duplicado, limite de armazenamento atingido ou informações inválidas.
- **Pós-condições:** ação cadastrada no vetor.

UC02 — Listar ações

- **Objetivo:** visualizar todas as ações cadastradas.
- **Ator:** usuário.
- **Pré-condições:** sistema em execução.
- **Fluxo principal:**
  1. O usuário seleciona a opção 2.
  2. O sistema verifica se existem ações cadastradas.
  3. O sistema apresenta os dados das ações.
- **Fluxo alternativo:** caso não existam ações, o sistema informa que não há registros.
- **Pós-condições:** informações exibidas ao usuário.

UC03 — Consultar ação

- **Objetivo:** localizar uma ação pelo código.
- **Ator:** usuário.
- **Pré-condições:** sistema em execução.
- **Fluxo principal:**
  1. O usuário seleciona a opção 3.
  2. O sistema solicita o código.
  3. O sistema procura o código no vetor.
  4. O sistema apresenta os dados encontrados.
- **Fluxo alternativo:** se o código não existir, o sistema informa que a ação não foi encontrada.
- **Pós-condições:** dados da ação consultados ou mensagem de inexistência apresentada.

UC04 — Atualizar situação

- **Objetivo:** alterar a situação de uma ação.
- **Ator:** usuário.
- **Pré-condições:** a ação deve estar cadastrada.
- **Fluxo principal:**
  1. O usuário seleciona a opção 4.
  2. O sistema solicita o código da ação.
  3. O sistema localiza a ação.
  4. O sistema apresenta as opções de situação.
  5. O usuário escolhe a nova situação.
  6. O sistema atualiza o registro.
  7. O sistema confirma a alteração.
- **Fluxo alternativo:** se a ação não existir, o sistema apresenta uma mensagem de erro.
- **Pós-condições:** situação atualizada.

UC05 — Registrar participação

- **Objetivo:** registrar a quantidade efetiva de participantes.
- **Ator:** usuário.
- **Pré-condições:** a ação deve estar cadastrada e possuir situação Realizada.
- **Fluxo principal:**
  1. O usuário seleciona a opção 5.
  2. O sistema solicita o código da ação.

3. O sistema verifica se a ação existe.
4. O sistema verifica se sua situação é Realizada.
5. O usuário informa a quantidade efetiva de participantes.
6. O sistema valida a quantidade.
7. O sistema registra a informação.

- **Fluxos alternativos:** ação inexistente, ação não realizada ou quantidade inválida.
- **Pós-condições:** quantidade efetiva armazenada.

UC06 — Gerar resumo

- **Objetivo:** apresentar informações gerais sobre as ações cadastradas.
- **Ator:** usuário.
- **Pré-condições:** sistema em execução.
- **Fluxo principal:**
  1. O usuário seleciona a opção 6.
  2. O sistema percorre as ações cadastradas.
  3. O sistema contabiliza as ações planejadas, realizadas e canceladas.
  4. O sistema calcula os totais de participantes.
  5. O sistema calcula o percentual de participação.
  6. O sistema apresenta o resumo.
- **Fluxo alternativo:** quando não existem participantes previstos nas ações realizadas, o percentual apresentado será zero.
- **Pós-condições:** resumo apresentado ao usuário.

4. Plano de testes e resultados

Esta tabela pode ser colocada diretamente no documento final. Os resultados devem ser preenchidos após executar os testes de verdade.

Teste	Procedimento	Resultado esperado
CT01	Cadastrar ação válida	Cadastro realizado
CT02	Cadastrar código repetido	Sistema rejeita
CT03	Listar ações	Exibir registros
CT04	Consultar código existente	Exibir ação
CT05	Consultar código inexistente	Informar inexistência
CT06	Atualizar situação	Situação alterada
CT07	Registrar participação em ação realizada	Quantidade registrada
CT08	Registrar participação em ação planejada	Operação rejeitada
CT09	Informar quantidade negativa	Sistema rejeita
CT10	Cadastrar mais de 100 ações	Sistema impede novo cadastro
CT11	Gerar resumo	Apresentar totais e percentual

CT12	Informar opção inválida	Sistema solicita uma opção válida
CT13	Escolher opção 0	Programa encerrado

5. Manual do usuário

MANUAL DO USUÁRIO

Sistema de Planejamento e Acompanhamento de Ações do Programa Saúde na Escola

1. Apresentação

O Sistema PSE foi desenvolvido com o objetivo de auxiliar na organização, no planejamento e no acompanhamento de ações coletivas do Programa Saúde na Escola.

A aplicação permite cadastrar, listar, consultar e atualizar ações, registrar a quantidade de participantes e gerar um resumo geral.

O programa foi desenvolvido em linguagem C e utiliza vetores para armazenar os registros durante sua execução.

2. Requisitos para utilização

Para executar o sistema, é necessário possuir:

- Computador com sistema operacional compatível.
- Compilador da linguagem C, como GCC.
- Terminal ou prompt de comando.

3. Compilação

Após baixar o código-fonte `sistema_pse.c`, abra o terminal na pasta em que o arquivo está localizado.

Utilize o seguinte comando:

bash

```
gcc sistema_pse.c -o sistema_pse
```

4. Execução

No Windows:

bash

```
sistema_pse.exe
```

No Linux:

bash

```
./sistema_pse
```

5. Utilização do sistema

Ao iniciar o programa, será apresentado o menu principal:

1. Cadastrar ação
2. Listar ações
3. Consultar ação
4. Atualizar situação
5. Registrar participação
6. Gerar resumo
7. Sair

O usuário deverá digitar o número correspondente à operação desejada.

6. Descrição das funcionalidades

Cadastrar ação: permite informar os dados de uma nova ação. Cada registro recebe um código único e inicia com situação Planejada.

Listar ações: apresenta todas as ações cadastradas durante a execução.

Consultar ação: permite localizar uma ação por meio de seu código.

Atualizar situação: permite alterar a situação para Planejada, Realizada ou Cancelada.

Registrar participação: permite informar a quantidade efetiva de participantes de uma ação realizada.

Gerar resumo: apresenta a quantidade de ações por situação, os totais de participantes e o percentual de participação.

Sair: encerra o programa.

7. Cuidados e limitações

O sistema utiliza dados fictícios e coletivos. Não devem ser inseridos nomes de estudantes, documentos pessoais, diagnósticos ou informações clínicas.

Os registros permanecem armazenados somente na memória durante a execução. Ao encerrar o programa, as informações são perdidas.

8. Considerações

A aplicação possui finalidade acadêmica e organizacional, não substituindo os profissionais envolvidos no Programa Saúde na Escola.

6. README.md para o GitHub

Sistema PSE — Saúde na Escola

Descrição

Projeto acadêmico desenvolvido para auxiliar no planejamento e acompanhamento de ações coletivas do Programa Saúde na Escola (PSE).

O sistema permite organizar informações sobre atividades planejadas, realizadas e canceladas, além de registrar dados coletivos de participação.

Objetivo

Desenvolver uma aplicação introdutória em linguagem C, aplicando conceitos de lógica de programação, estruturas condicionais, repetição, vetores e funções.

Integrantes

- Juan Guimarães Mendonça
- Murilo Constancio Amadeu
- Vitor Garrido da Silva

Tecnologias utilizadas

- Linguagem C
- Compilador GCC
- GitHub

Funcionalidades

1. Cadastro de ações.
2. Listagem de ações.
3. Consulta por código.
4. Atualização de situação.
5. Registro de participação.
6. Geração de resumo.

Como executar

Compile o programa:

bash

```
gcc sistema_pse.c -o sistema_pse
```

Execute no Windows:

bash

```
sistema_pse.exe
```

Execute no Linux:

bash

```
./sistema_pse
```

Organização

text

```
projeto-pse/
│
├── README.md
├── src/
│   └── sistema_pse.c
├── documentacao/
│   ├── documento-final.pdf
│   ├── manual-do-usuario.pdf
│   └── casos-de-uso.pdf
│
└── testes/
    └── casos-de-teste.txt
```

Armazenamento

Os dados são armazenados em vetores durante a execução do programa. Não existe persistência após o encerramento.

Privacidade

O projeto utiliza dados fictícios e coletivos. Não armazena informações individuais ou clínicas de estudantes.

Finalidade

Projeto acadêmico do curso de Engenharia de Software — UNICESUMAR, 2026.